

```
<!--Joaquin, Gabriel y Jedermin -->
```

Selección de Proyecto {

```
<Var Nuevo_Proyecto String/>
```

}



Opciones:

- 01 Sistema de Gestión de Correos Electrónicos
- 02 **Sistema de Chat**
- 03 Sistema de Gestión de imágenes
- 04 Sistema de Gestión de Videos

Introducción {

La comunicación por mensajes es una de las formas más utilizadas hoy en día para interactuar entre personas.

Aplicaciones como chats y redes sociales permiten enviar información de manera casi instantánea, pero detrás de esto existe un proceso técnico que hace posible esta comunicación.

En este proyecto se busca desarrollar un sistema de chat básico para entender cómo funciona este proceso.

- Comunicación entre usuarios
- Envío y recepción de mensajes
- Procesamiento de datos
- Uso de arquitectura cliente-servidor
- Intercambio de información en red
- Almacenamiento de mensajes
- Funcionamiento básico de un chat

}

Objetivos {

Desarrollo del sistema de chat

Desarrollar un sistema de chat básico que permita a los usuarios comunicarse mediante el envío y recepción de mensajes.

- Comunicación entre usuarios
- Envío de mensajes
- Recepción de mensajes
- Uso de cliente-servidor
- Manejo de datos
- Implementación básica

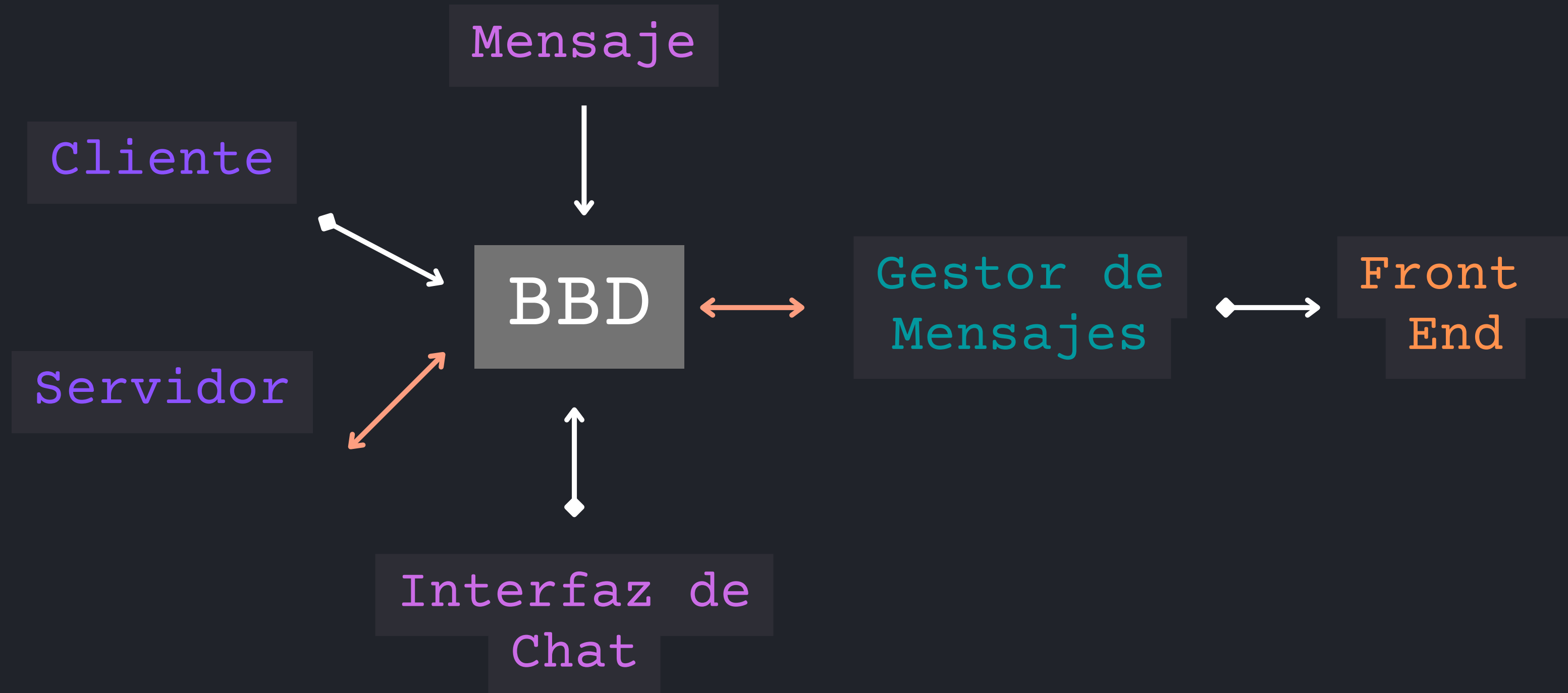
Comprensión del funcionamiento

Entender cómo funciona la arquitectura cliente-servidor y el manejo de datos dentro de una aplicación.

}

Estructura {

La estructura del sistema muestra cómo los diferentes componentes, como el cliente, el servidor, la base de datos y el gestor de mensajes, interactúan para permitir el envío, procesamiento y almacenamiento de los mensajes.



}

Funcionalidades {

Funciones del Sistema

El sistema permite a los usuarios comunicarse mediante el envío y recepción de mensajes, facilitando la interacción dentro de la aplicación de forma sencilla.

- Registro de usuario
- Envío de mensajes
- Recepción de mensajes
- Visualización de conversaciones
- Almacenamiento de mensajes

Características Principales

Estas funcionalidades permiten que el sistema funcione de manera organizada, asegurando que los mensajes se envíen, procesen y almacenen correctamente.

- Interfaz simple
- Comunicación rápida
- Gestión de mensajes
- Flujo de información
- Base para futuras mejoras

}

Arquitectura {

El sistema se basa en una arquitectura cliente-servidor, donde múltiples clientes se conectan a un servidor central.

El cliente es la parte visual con la que interactúa el usuario, mientras que el servidor es el núcleo del sistema, encargado de procesar los mensajes, gestionar usuarios y distribuir la información.

Para la comunicación se utiliza WebSockets, lo cual permite mantener una conexión persistente entre cliente y servidor, logrando comunicación en tiempo real.

Cliente

- Interfaz de usuario
- Envá y recibe mensajes

Servidor

- Procesamiento de mensajes
- Gestión de usuarios
- Distribución en tiempo real

Comunicación

- WebSocket

}

Tecnología {

Para el desarrollo del sistema se seleccionaron tecnologías enfocadas en rendimiento, escalabilidad y comunicación en tiempo real.

Como lenguaje principal se seleccionó Go debido a su eficiencia en el manejo de concurrencia, lo que permite atender a varios usuarios al mismo tiempo sin afectar el rendimiento.

Para la comunicación en tiempo real se utiliza WebSockets, ya que permiten mantener una conexión activa entre cliente y servidor, lo cual es fundamental para aplicaciones de chat donde los mensajes deben enviarse y recibirse instantáneamente.

Para el formato de datos se va a utilizar JSON porque permite estructurar de forma clara y estándar, facilitando el intercambio entre sistemas

Finalmente se utilizarán herramientas como Git y GitHub para el control de versiones y trabajo colaborativo



}

Paquetes {

Paquetes necesarios

Durante la fase de planeación se investigaron los paquetes necesarios para implementar el sistema.

- El paquete `net/http` permite crear el servidor que recibirá las conexiones.
- Gorilla WebSocket es una librería externa que facilita la implementación de comunicación en tiempo real.
- `encoding/json` permite estructurar los mensajes que se envían entre cliente y servidor.
- `Sync` permite manejar múltiples usuarios de forma segura, evitando conflictos en memoria.



}

Conclusión

01

Revisión
continua del
código.

02

Optimizar
las Apis de
proveedores.

03

Corregir
bugs.

04

Optimizar el
rendimiento
general.

05

Organizar al
equipo.

06

Distribuir
la faena por
bloques.

07

Subir los
cambios a la
vez.

08

Limpiar la
base de
datos.

}

```
<!--Estudio Shonos-->
```

Gracias {

```
<fmt.Println("Empezamos")/>
```

}